

CẨM NANG VẤN ĐÁP - THÀNH VIÊN 3: KIỂM THỬ THỦ CÔNG & VÒNG ĐỜI (Đóng góp 15%)

Dự án: Phần mềm Học từ vựng (LearnFlash)

Nhiệm vụ: Kiểm thử thủ công, viết hàm Validation cơ bản & giám sát Vòng đời Activity



PHẦN 1: MÔ TẢ CÔNG VIỆC ĐÃ LÀM (Manual Testing & Lifecycle Logging)

Ở mức độ đóng góp 15%, công việc của bạn tập trung hoàn toàn vào việc chạy kiểm thử thủ công để đảm bảo ứng dụng không lỗi, kiểm tra dữ liệu đầu vào đơn giản và theo dõi vòng đời chạy của ứng dụng, bao gồm:

1. **Kiểm thử thủ công (Manual Testing):** Chạy ứng dụng trên thiết bị thật/máy ảo để test các tính năng như thêm/sửa từ vựng, ôn tập thẻ, chuyển đổi theme sáng/tối, xuất nhập file.
2. **Ràng buộc dữ liệu cơ bản (Validation):** Viết điều kiện kiểm tra xem ô nhập liệu từ khóa hoặc ý nghĩa có bị để trống hay không trước khi bấm lưu.
3. **Ghi log Vòng đời (Lifecycle Logging):** Override 6 callback vòng đời cơ bản trong MainActivity và gọi hàm `Log.d` ghi log để quan sát thứ tự chạy trong Android Studio Logcat.



Các file liên quan trực tiếp:

- [MainActivity.kt](#) (Gồm 6 hàm callback ghi log vòng đời)
- [ChiTietViewModel.kt](#) (Chứa logic check trống từ khóa/ý nghĩa)



PHẦN 2: CÂU HỎI VẤN ĐÁP THƯỜNG GẶP (Rút gọn)

? Câu 1: Bạn đã override các callback vòng đời nào trong MainActivity và theo thứ tự nào khi xoay màn hình?

- **Trả lời:**
 - Các callback đã override: `onCreate()`, `onStart()`, `onResume()`, `onPause()`, `onStop()`, `onDestroy()`.
 - Khi **xoay màn hình**, Activity cũ bị hủy hoàn toàn và Activity mới được tạo lại. Logcat in ra theo thứ tự:
 1. `onPause` : Activity tạm dừng nhận sự kiện.
 2. `onStop` : Giao diện cũ bị ẩn đi.
 3. `onDestroy` : Activity cũ bị hủy khỏi RAM.
 4. `onCreate` : Activity mới được khởi tạo và nạp lại UI Compose.

- 5. onStart : Giao diện mới hiển thị trở lại.
- 6. onResume : Sẵn sàng tương tác.
- (Bạn đã ghi log với tag bộ lọc Logcat là LearnFlash_Lifecycle để theo dõi các sự kiện này).

? Câu 2: Làm thế nào bạn đảm bảo dữ liệu trong ViewModel không bị mất khi xoay màn hình?

- **Trả lời:**
 - Dữ liệu được lưu trong **ViewModel**. Vòng đời của ViewModel dài hơn Activity và sống sót qua sự kiện thay đổi cấu hình (xoay màn hình).
 - Khi Activity bị hủy và tạo lại do xoay màn hình, hệ thống sẽ gán lại chính instance ViewModel cũ cho Activity mới. Do đó, các biến State lưu danh sách từ vựng, từ đang học... vẫn được giữ nguyên vẹn mà không cần tải lại từ Database.

? Câu 3: Bạn thực hiện kiểm tra tính hợp lệ dữ liệu (Validation) và hiển thị thông báo lỗi như thế nào?

- **Trả lời:**
 - **Kiểm tra:** Trong ChiTietViewModel.kt, trước khi lưu từ vựng mới, ta check điều kiện: `if (_tuKhoa.value.isBlank())` hoặc `if (_nghiaTiengViet.value.isBlank())`. Nếu đúng, gán thông báo lỗi vào biến State `loiNhapLieu`.
 - **Hiển thị trên UI:** Ở file giao diện ChiTietUI.kt, ô nhập `OutlinedTextField` liên kết thuộc tính `isError` với biến trạng thái lỗi trên. Khi lỗi xảy ra, đường viền ô nhập tự động đổi sang màu đỏ để cảnh báo người dùng.

? Câu 4: Hãy nêu một kịch bản kiểm thử (Test Case) thử công quan trọng bạn đã thực hiện để kiểm tra tính năng Offline?

- **Trả lời:**
 - **Kịch bản:** Test tính năng Offline (Offline-first).
 - **Các bước thực hiện:**
 1. Mở ứng dụng lần đầu khi có mạng để tải dữ liệu mẫu về máy.
 2. Tắt hoàn toàn kết nối Wifi/4G của điện thoại.
 3. Thực hiện thêm từ vựng mới và học thử ôn tập.
 - **Kết quả:** Ứng dụng vẫn chạy mượt mà, thêm từ và ôn tập bình thường vì mọi thao tác ghi dữ liệu đều được lưu trực tiếp vào SQLite cục bộ thông qua Room DB. Khi có mạng trở lại, Firestore sẽ tự động đồng bộ phần dữ liệu mới lên cloud.